Gradient Descent: Intuition, Mathematics, and Examples

DAEN 300

Fall 2026

Learning objectives

1. Explain why following a negative derivative reduces a function locally.
2. Apply the gradient descent update rule by hand.
3. Describe how learning rate affects stability and speed.
4. Derive and apply gradient descent for simple linear regression.
5. Distinguish full-batch and stochastic gradient descent.

The optimization problem

We have an objective (or loss) function $J(\theta)$, and we want parameters that make it small:

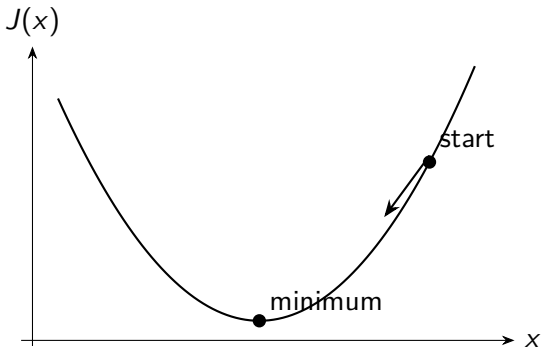
$$\theta^* \in \operatorname*{argmin}_{\theta} J(\theta).$$

- In regression, θ contains coefficients of a prediction rule.
- The loss measures how far predictions are from observed values.
- A direct algebraic solution is not always available or convenient.
- Gradient descent uses many small, informed updates instead.

Intuition: walking downhill

Imagine standing on a curved hillside with limited visibility.

- Measure the slope at your current location.
- If the slope is positive, move to the left; if negative, move right.
- On a flat spot, the slope is zero, but this need not be the global minimum.
- Repeat, taking steps scaled by how steep the slope is.



Why the negative derivative?

For a small change Δx, a differentiable function obeys

$$J(x + \Delta x) \approx J(x) + J'(x)\Delta x.$$

Take $\Delta x = -\eta J'(x)$ with $\eta > 0$:

$$J(x + \Delta x) - J(x) \approx -\eta [J'(x)]^2 \leq 0.$$

- The derivative supplies the *direction* and local steepness.
- The learning rate η controls the step size.
- This predicts a decrease for sufficiently small steps; a large step can increase loss.

The update rule

$$\boxed{x_{t+1} = x_t - \eta J'(x_t)}$$

x_t	current location after t updates
$J'(x_t)$	slope at the current location
$\eta > 0$	learning rate (step-size multiplier)
x_{t+1}	updated location

For a parameter vector $\boldsymbol{\theta}$:

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla J(\boldsymbol{\theta}_t)}.$$

Worked example 1: a parabola

Consider the illustrative function also used in Nield's discussion:

$$J(x) = (x-3)^2 + 4, \qquad J'(x) = 2(x-3).$$

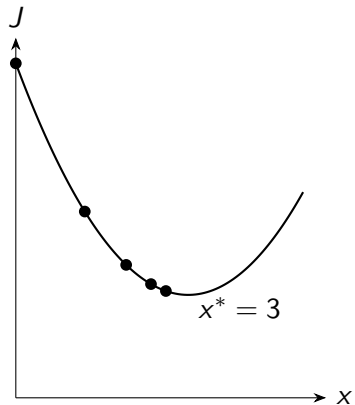
Its minimum is $x^* = 3$, with $J(x^*) = 4$. Start at $x_0 = 0$ and choose $\eta = 0.2$.

$$x_{t+1} = x_t - 0.2 \cdot 2(x_t - 3) = 0.6x_t + 1.2.$$

t	x_t	$J'(x_t)$	$J(x_t)$
0	0.0000	-6.0000	13.0000
1	1.2000	-3.6000	7.2400
2	1.9200	-2.1600	5.1664
3	2.3520	-1.2960	4.4199
4	2.6112	-0.7776	4.1512

What the example shows

- The starting slope is -6; subtracting a negative quantity moves x to the right.
- As x approaches 3, $|J'(x)|$ shrinks and steps become shorter.
- The iterates approach 3, though a finite run may not reach it exactly.



Learning rate: too small, useful, too large

For $J(x) = (x - 3)^2 + 4$, let $e_t = x_t - 3$.
Then

$$e_{t+1} = (1 - 2\eta)e_t.$$

Learning rate	Error multiplier	Consequence
$\eta = 0.05$	0.90	converges slowly
$\eta = 0.20$	0.60	converges smoothly
$\eta = 0.75$	-0.50	oscillates but converges
$\eta = 1.00$	-1.00	oscillates without convergence
$\eta = 1.20$	-1.40	diverges

For this particular quadratic, convergence from any starting x_0 occurs exactly when $0 < \eta < 1$.

From one variable to many

A regression line has two trainable parameters:

$$\widehat{y} = mx + b, \qquad \boldsymbol{\theta} = \begin{bmatrix} m \\ b \end{bmatrix}.$$

The loss is a *surface* over (m, b), not a curve over x.

$$\nabla J(m, b) = \begin{bmatrix} \partial J/\partial m \\ \partial J/\partial b \end{bmatrix}.$$

Update both parameters simultaneously using gradients calculated at the same current point:

$$\boxed{m_{t+1} = m_t - \eta \frac{\partial J}{\partial m}}, \qquad \boxed{b_{t+1} = b_t - \eta \frac{\partial J}{\partial b}}.$$

Choosing a regression loss

Given observations (x_i, y_i), let prediction errors be

$$r_i = mx_i + b - y_i.$$

We minimize the mean squared error with a convenient factor of $1/2$:

$$\boxed{J(m, b) = \frac{1}{2n} \sum_{i=1}^{n} (mx_i + b - y_i)^2}.$$

- Squaring makes both positive and negative errors costly.
- Larger errors receive a larger penalty.
- The factor $1/2$ cancels the 2 introduced by differentiation.
- This scaling has the same minimizer as ordinary sum of squared errors.

Deriving the partial derivatives

Use the chain rule for each squared residual:

$$\frac{\partial}{\partial m}\frac{(mx_i + b - y_i)^2}{2n} = \frac{1}{n}(mx_i + b - y_i)x_i.$$

Similarly, the derivative with respect to b multiplies by 1:

$$\boxed{\frac{\partial J}{\partial m} = \frac{1}{n}\sum_{i=1}^{n}(mx_i + b - y_i)x_i}$$

$$\boxed{\frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(mx_i + b - y_i)}.$$

Interpretation: the slope parameter reacts to error weighted by x_i; the intercept reacts to the average signed error.

Worked example 2: fitting a line

A deliberately small dataset allows every calculation to be checked by hand:

$$(x_1, y_1) = (0, 1), \qquad (x_2, y_2) = (1, 3).$$

Start with $m_0 = 0$, $b_0 = 0$, and $\eta = 0.2$.

i	x_i	y_i	$\widehat{y}_i$	r_i
1	0	1	0	-1
2	1	3	0	-3

$$\frac{\partial J}{\partial m} = \frac{(-1)(0) + (-3)(1)}{2} = -1.5, \qquad \frac{\partial J}{\partial b} = \frac{-1 - 3}{2} = -2.$$

First regression update: compute and check

Gradient descent changes both parameters:

$$m_1 = 0 - 0.2(-1.5) = 0.3, \qquad b_1 = 0 - 0.2(-2) = 0.4.$$

The fitted prediction rule after one update is

$$\boxed{\widehat{y} = 0.3x + 0.4}.$$

Initial loss:

$$J(0,0) = \tfrac{1}{4}\left[(-1)^2 + (-3)^2\right] = 2.5.$$

Updated loss:

$$J(0.3, 0.4) = \tfrac{1}{4}\left[(-0.6)^2 + (-2.3)^2\right] = 1.4125.$$

The loss decreases in one iteration. Continued updates approach the exact line $y = 2x + 1$.

An executable Python implementation

```
points = [(0.0, 1.0), (1.0, 3.0)]
m, b = 0.0, 0.0
eta = 0.2

for step in range(200):
   n = len(points)
   dm = sum((m*x + b - y)*x for x, y in points)/n
   db = sum((m*x + b - y) for x, y in points)/n
   m -= eta*dm
   b -= eta*db

print(m, b) # close to (2, 1)
```

- Compute `dm` and `db` *before* updating either parameter.
- The chosen learning rate works for this illustrative dataset, not necessarily for arbitrary datasets.

Batch versus stochastic gradient descent

	Full-batch GD	Stochastic GD (SGD)
Gradient uses	Every training example	One randomly selected example
Updates	One per full pass	Many within each pass
Trajectory	Relatively smooth	Noisier
Typical benefit	Stable gradient estimate	Lower cost per update; useful at scale

- **Mini-batch GD** uses a small set of examples per update.
- With SGD, a decaying learning rate can help reduce fluctuation near a minimum.

When does gradient descent stop?

Practical stopping criteria include:

- Gradient magnitude becomes small: $\|\nabla J(\boldsymbol{\theta}_t)\| < \varepsilon$.
- Improvement in loss becomes negligible.
- A preset iteration or compute budget is reached.
- Validation error stops improving (important in predictive modeling).

Cautions: A zero gradient may indicate a minimum, maximum, or saddle point; nonconvex losses may have multiple local minima. Feature scaling may improve optimization behavior.

Classroom check: calculate the next step

Let $J(x) = (x-3)^2 + 4$, with $x_0 = 5$ and $\eta = 0.25$.

1. What is the derivative at x_0?
2. What is x_1?
3. Compare $J(x_0)$ and $J(x_1)$.

Solution: $J'(5) = 4, \quad x_1 = 5 - 0.25(4) = 4, \quad J(5) = 8, \quad J(4) = 5.$

Key takeaways

1. A derivative (or gradient) identifies the direction of steepest local increase.
2. Subtracting a scaled gradient moves toward lower loss when the step is sufficiently small.
3. Learning rate determines how quickly and reliably the method progresses.
4. Linear regression uses the same logic to update slope and intercept.
5. Iterative optimization generalizes far beyond simple regression.